Experiment-6
Hamming Code

AIM:

To implement error detection and correction using Hamming Code and verify the error correction capability using a test data stream.

PROCEDURE:

Sender

1. Create a sender program.
2. Read a text message from the input file.
3. Convert the text into binary.
4. Calculate and insert the required redundant parity bits.
5. Generate the Hamming Code.
6. Store the encoded data in a file named Channel.

Receiver:

7. Read the encoded data from channel.
8. Recalculate the parity bits.
9. Determine whether an error is present.
10. If an error is detected, find its position.
11. Correct the erroneous bit.

CODE:-

```
def hamming_encode(data):
    encoded = ""

    for i in range(0, len(data), 4):
        d = data[i:i+4]

        if len(d) < 4:
            d = d.ljust(4, '0')

        d1, d2, d3, d4 = map(int, d)

        p1 = d1 ^ d2 ^ d4
        p2 = d1 ^ d3 ^ d4
        p4 = d2 ^ d3 ^ d4
```

```
    code = str(p1) + str(p2) + str(d1) + str(p4) + str(d2) +
                                                str(d3) + str(d4)
    encode += code
  return encoded
text = input("Enter text: ")
binary = ''.join(format(ord(c), '08b') for c in text)
Print("Binary data:", binary)
encoded_data = hamming_encode(binary)
with open("channel.txt", "w") as file:
    file.write(encoded_data)
Print("Hamming encoded data:", encoded_data)
Print("Data saved to channel.txt")
```

12. Remove the redudant bits.
13. Convert the corrected binary data back into ASCII/text.
14. Display the recovered message.

These sender/receiver requirements are directly specified in Practical-6 of the manual.

Sample Output:

Enter text: HELLO

Binary data:

01001000 01000101 01001100 01001100 01001111

Hamming Code:

...

Error detected at position: 12

Error corrected successfully.

Decoded Data:

HELLO.

Result:

Thus, Hamming Code was successfully implemented to detect and correct a single-bit error in the transmitted data.